UNIT - 3
BOOLEAN ALGEBRA

- It is an algebra that deals with binary number systems.
- it is useful in designing logic circuits, which are used by the processors of computer systems.
- George Boole (1815- 1864), an English mathematician, developed it for Simplifying representation Manipulation of propositional logic In 1938,
- Claude E Shannon proposed using Boolean algebra in design of relay switching circuits it Provides economical and straightforward approach.

Boolean Algebra:

Boolean Algebra is a branch of algebra that deals with boolean values—true and false. It is fundamental to digital logic design and computer science, providing a mathematical framework for describing logical operations and expressions

Boolean Algebra Operations

The basic operations of Boolean algebra are as follows:

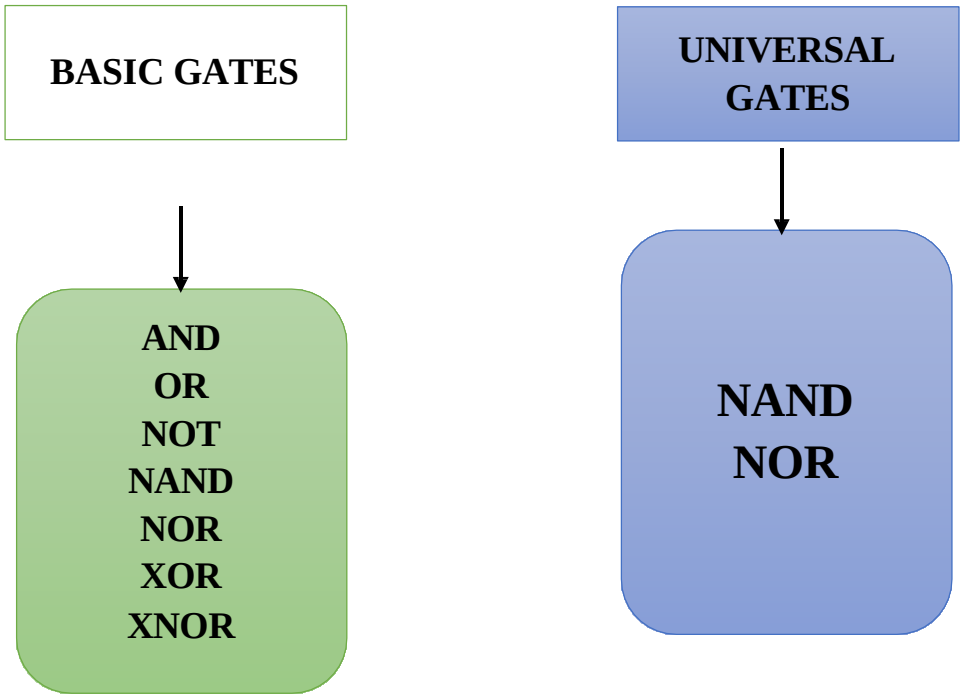
- Conjunction or AND operation
- Disjunction or OR operation
- Negation or Not operation

Below is the table defining the symbols for all three basic operations.

Operator	Symbol	Precedence
NOT	‘ (or) ¬	Highest
AND	. (or) ∧	Middle
OR	+ (or) ∨	Lowest

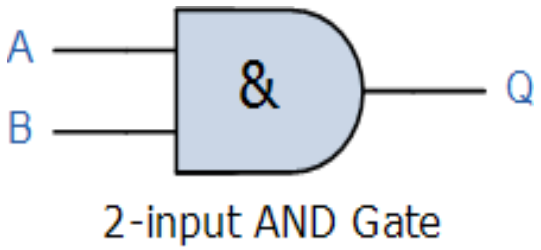
It is an algebra that deals with binary number systems.it is useful in designing logic circuits, which are used by the processors of computer systems. George Boole (18151864), an English mathematician, developed it for Simplifying Representation Manipulation of propositional logic In 1938,
Claude E Shannon proposed using Boolean algebra in design of relay switching circuits it Provides economical and straightforward approach

FUNDAMENTALS OF GATES



1) AND GATE

For a 2-input AND gate, the output Q is true if BOTH input A "AND" input B are both true, giving the Boolean Expression of: (Q = A . B)

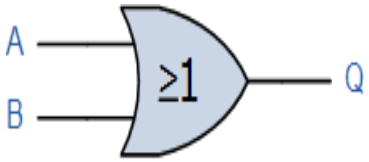


2-input AND Gate

AND Truth Table		
A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

2) OR GATE

For a 2-input OR (Inclusive-OR) gate, the output Q is true if EITHER input A "OR" input B is true, giving the Boolean Expression of: (Q = A + B)

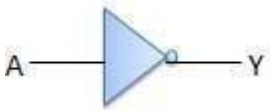


2-input OR Gate

Inputs		Output
A	B	A + B
0	0	0
0	1	1
1	0	1
1	1	1

3) NOT GATE

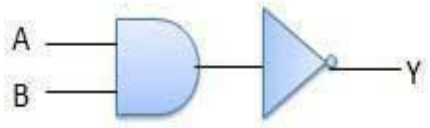
For a single input NOT (Inverter) gate, the output Q is ONLY true when the input is "NOT" true, the output is the inverse or complement of the input giving the Boolean Expression of: (Q = NOT A).

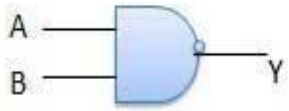


NOT Truth Table	
A	Q
0	1
1	0

4) NAND GATE

For a 2-input NAND gate, the output Q is NOT true if BOTH input A and input B are true, giving the Boolean Expression of: (Q = not(A AND B)).

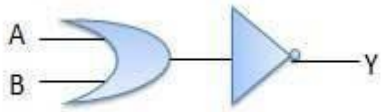


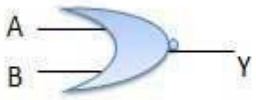


Inputs		Output
A	B	$\overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

5) NOR GATE

For a 2-input NOR gate, the output Q is true if BOTH input A and input B are NOT true, giving the Boolean Expression of: (Q = not(A OR B)).

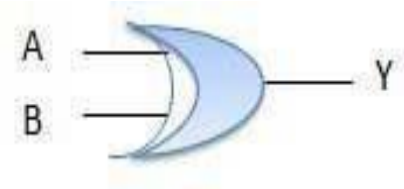




Inputs		Output
A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

6) X-OR

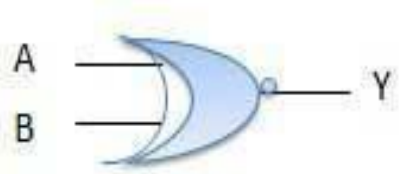
For a 2-input Ex-OR gate, the output Q is true if EITHER input A or if input B is true, but NOT both giving the Boolean Expression of: (Q = (A and NOT B) or (NOT A and B)).



Inputs		Output
A	B	A ⊕ B
0	0	0
0	1	1
1	0	1
1	1	0

7) X-NOR

For a 2-input Ex-NOR gate, the output Q is true if BOTH input A and input B are the same, either true or false, giving the Boolean Expression of: (Q = (A and B) or (NOT A and NOT B))



Inputs		Output
A	B	A ⊙ B
0	0	1
0	1	0
1	0	0
1	1	1

Note:

Inputs		Truth Table Outputs For Each Gate					
A	B	AND	NAND	OR	NOR	EX-OR	EX-NOR
0	0	0	1	0	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	0	1	0	0	1

Logic Function	Boolean Notation
AND	A.B
OR	A+B
NOT	$\overline{A}$
NAND	$\overline{A.B}$
NOR	$\overline{A+B}$
EX-OR	$(A.\overline{B}) + (\overline{A}.B)$ or $A \oplus B$
EX-NOR	$(A.B) + (\overline{A}.\overline{B})$ or $\overline{A \oplus B}$

UNIVERSAL GATES

The universal gates are the logic gates that are versatile in that they can be programmed to execute any Boolean function. The most popular types among them are **NAND and NOR** gates.

BASIC LAWS

Identity Law

This law states that a variable would remain unchanged when it is ANDed with '1' or ORed with '0', i.e.,

- X.1 = X
- X + 0 = X

It states that a variable that has been ANDed with 0 gives us a 0, while a variable that has been ORed with 1 gives us a 1, i.e.,

- X.0 = 0
- X + 1 = 1

Idempotent Law

This law states that a variable would remain unchanged when it is ANDed or ORed with itself, i.e.,

- X + X = X
- X.X = X

Complement Law

This law states that in case a complement is added to any variable, then it would give one, whereas when we multiply this variable with its own complement, then it would result in '0', i.e.,

- X + X' = 1
- X.X' = 0

Commutative Law

The order of the variable or two different terms does not matter according to this law. It can be represented as follows,

- X + Y = Y + X
- X.Y = Y.X

Associative Law

According to this law, the order of an operation does not matter when the priority of the given variables are similar, such as '*' and '/', i.e.,

- X + (Y + Z) = (X + Y) + Z
- X.(Y.Z) = (X.Y).Z

Distributive Law

Using this law, we try to understand how the opening up of the brackets available in an operation, i.e.,

- X.(Y+Z) = (X.Y) + (X.Z)
- X + (Y.Z) = (X + Y).(X + Z)

Absorption Law

Using this law, we absorb similar variables, i.e.,

- X.(X + Y) = X
- X + XY = X

De Morgan's Theorem

In Boolean algebra, it gives the relation between AND, OR, and complements of the variable, and in logic circuits.

De Morgan's Law in Boolean Algebra

De Morgan's Law Boolean Algebra defines the relation between the OR, AND, and the complements of variables, and is given for both the complement of AND and OR of two values. In Boolean Algebra there are two De Morgan's Laws that are:

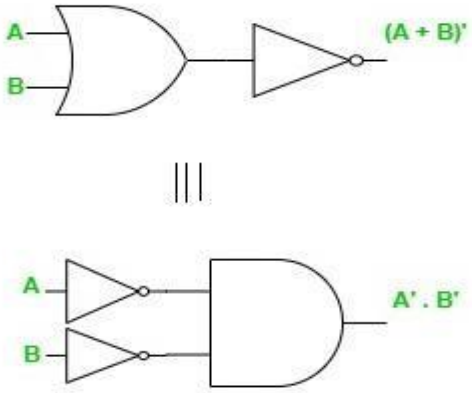
First De Morgan's Law

Second De Morgan's Law

First De Morgan's Law in Boolean Algebra

First De Morgan's law states that "The complement of OR of two or more variables is equal to the AND of the complement of each variable."
Let A and B be two variables, then mathematically First De Morgan's Law is given as: (A + B)' = A' . B'
Where

- + represents the OR operator between variables,
- . represents AND operator between variables, and
- ' represents complement operation on variable.



First De Morgan's Law Truth Table

A	B	A + B	(A + B)'	A'	B'	A'. B'
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

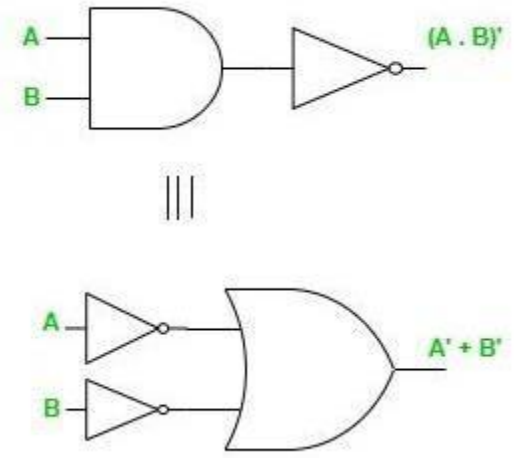
The truth table for first De Morgan's Law is given as follows:

Second De Morgan's Law in Boolean Algebra

Second De Morgan's law states that "The complement of AND of two or more variables is equal to the OR of the complement of each variable."
Let A and B be two variables, then mathematically Second De Morgan's Law is given as: (A . B)' = A' + B'
Where

- + represents the OR operator between variables,
- . represents AND operator between variables, and
- ' represents complement operation on variable.

Second De Morgan's Law Logic Gates



Second De Morgan's Law Truth Table

A	B	A . B	(A. B)'	A'	B'	A' + B'
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

The truth table for the second De Morgan's Law is given as follows:

Duality Theorem

The Dual of the Boolean function can be easily obtained by interchanging the logical AND operator with the logical OR operator and the zeros with ones and vice versa.

Note:

We can interchange 0 with 1,
1 with 0,
(+) sign with (.) sign
(.) sign with (+) sign to perform dual operation.

Examples:

Given Expression	Dual Expression
0=1	1=0
0.1=0	1+0=1
A.0=0	A+1=1
A.B=B.A	A+B=B+A
A.A=0	A+A=1
A.(B.C)=(A.B).C	A+(B+C)=(A+B)+C
AB=A+B	A+B=A.B
(A+B)(A+C)=AB+AC	AB+AC= (A+B)(A+C)

MINTERM AND MAXTERM

What is Minterm?

Minterm is the product of various different literals in which each literal occurs exactly once. The output result of the minterm functions is 1. It is represented by m. To represent a function, we perform a sum of minterms also called the **Sum Of Products (SOP)**.

Table for Two-Variable Minterm

Variable		Minterm	
A	B	Term	Representation
0	0	A’B’	m0
0	1	A’B	m1
1	0	AB’	m2
1	1	AB	m3

What is Maxterm?

Maxterm is the sum of various different literals in which each literal occurs exactly once. The output result of the maxterm functions is 0. It is represented by M. To represent a function, we perform product of maxterms also called **Product of Sum (POS).**

Table for Two Variable Maxterm

Variable		Maxterm	
A	B	Term	Representation
0	0	A+B	M0
0	1	A+B’	M1
1	0	A’+B	M2
1	1	A’+B’	M3

SOP AND POS

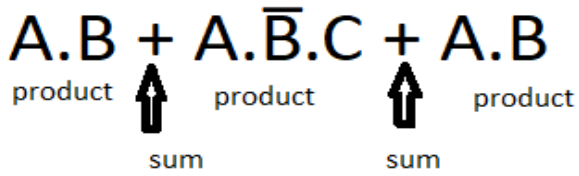
In Digital Electronics any logic circuit’s output is the function of digital inputs and the relation between input and output can be represented using logic table or Boolean expressions. This Boolean expression can be represented in two forms.

1. **Sum of Product (SOP)**
2. **Product of Sum (POS)**

Sum of Product Form(SOP)

In the sum of the product form of representation, The product num is logical and operation of the different input variables where the variables could be in the true form or in the complemented form.

The Sum of Product (SOP) form refers to a Boolean expression where several product terms (involving AND operations) are added up.



1. Non-Canonical SOP Form

In this form each product term between may or may not contain all the variables of the function.

Example: $F(A,B,C) = A + \bar{B}.C + A.C$

as we can see in above example the function have variables A, B, C but we are not including each variable in each product term. in first product term (A) we have not included B & C. In second product term we ($\bar{B}$ .C) have not included A. While in last product term we have not included B.

2. Canonical SOP Form

In canonical SOP form each product term contains all the variables of the function, where variables in each product term can be in true form or complemented form.

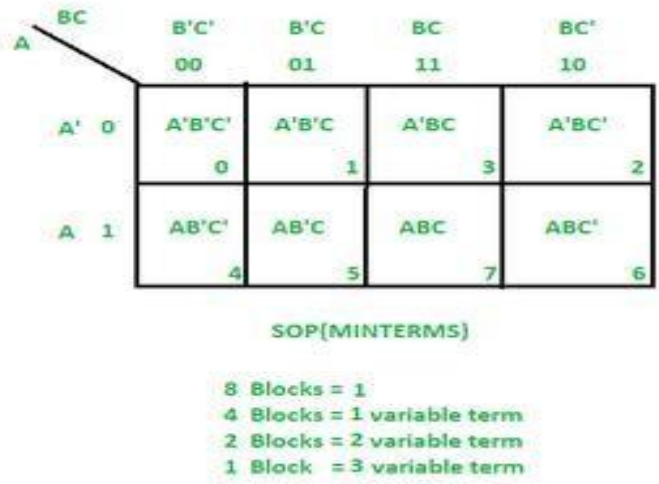
Example : $F(A,B) = \bar{A}.B + A.\bar{B}$

As we can see in above example each product term contain all the variables which are present in function. In first one($\bar{A}$.B) A is present in complementary form while B is in true form. In second one A is present in true form while B is in complementary form.

SOP FORM

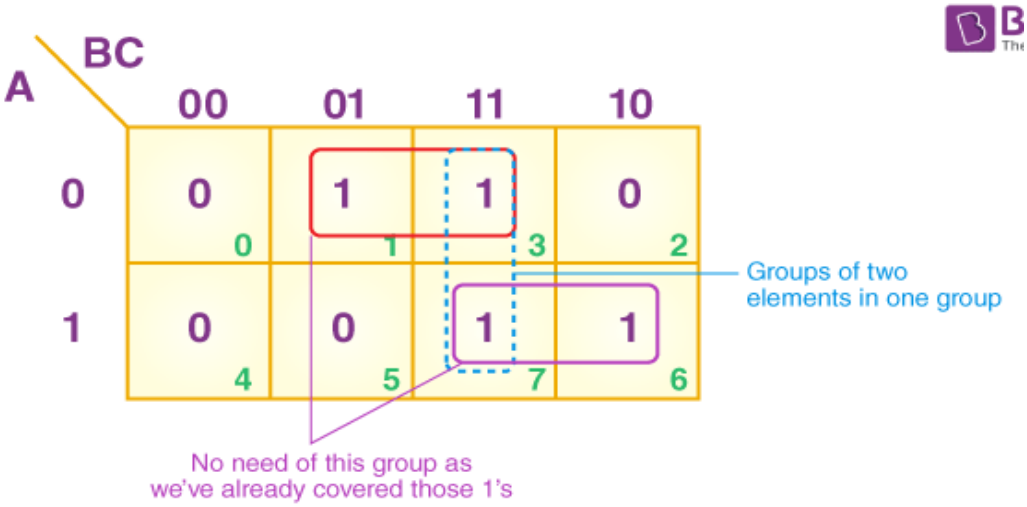
3 variables K-map

General Syntax :



Examples:

1) Z = ∑P, Q, R (1, 3, 6, 7)

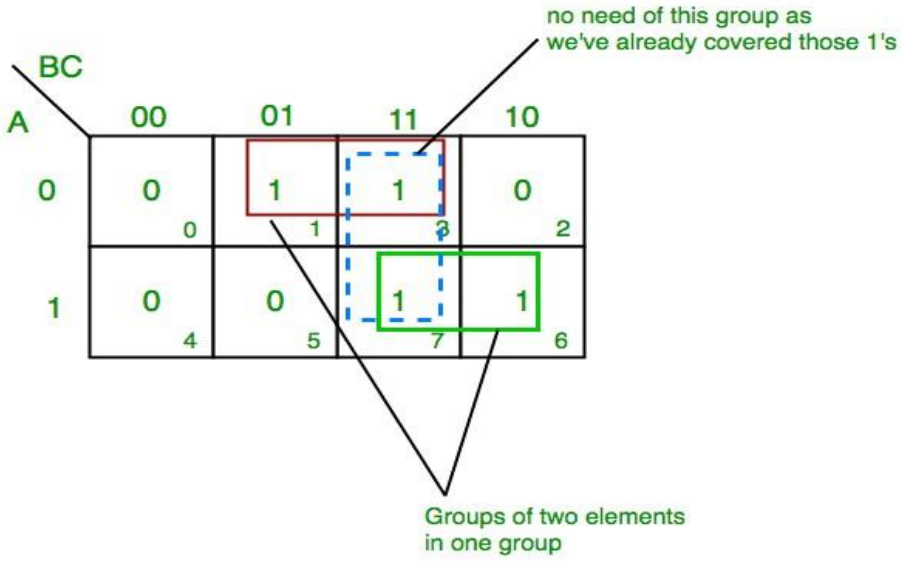


From the red group, the product term would be — P'R

From the green group, the product term would be — PQ

If we sum these product terms, then we will get this final expression (P'R + PQ)

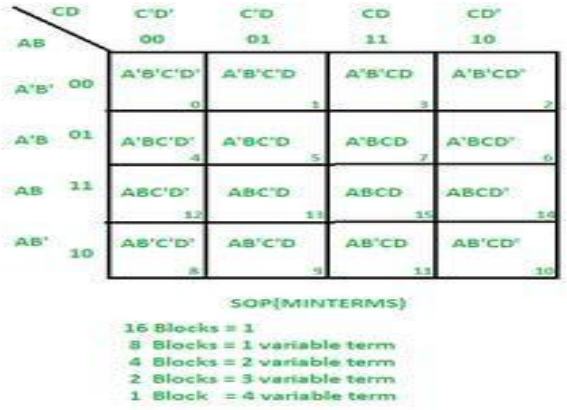
2) Z= ∑A,B,C(1,3,6,7)



From red group we get product term— A'C
From green group we get product term— AB
Summing these product terms we get- Final expression (A'C+AB)

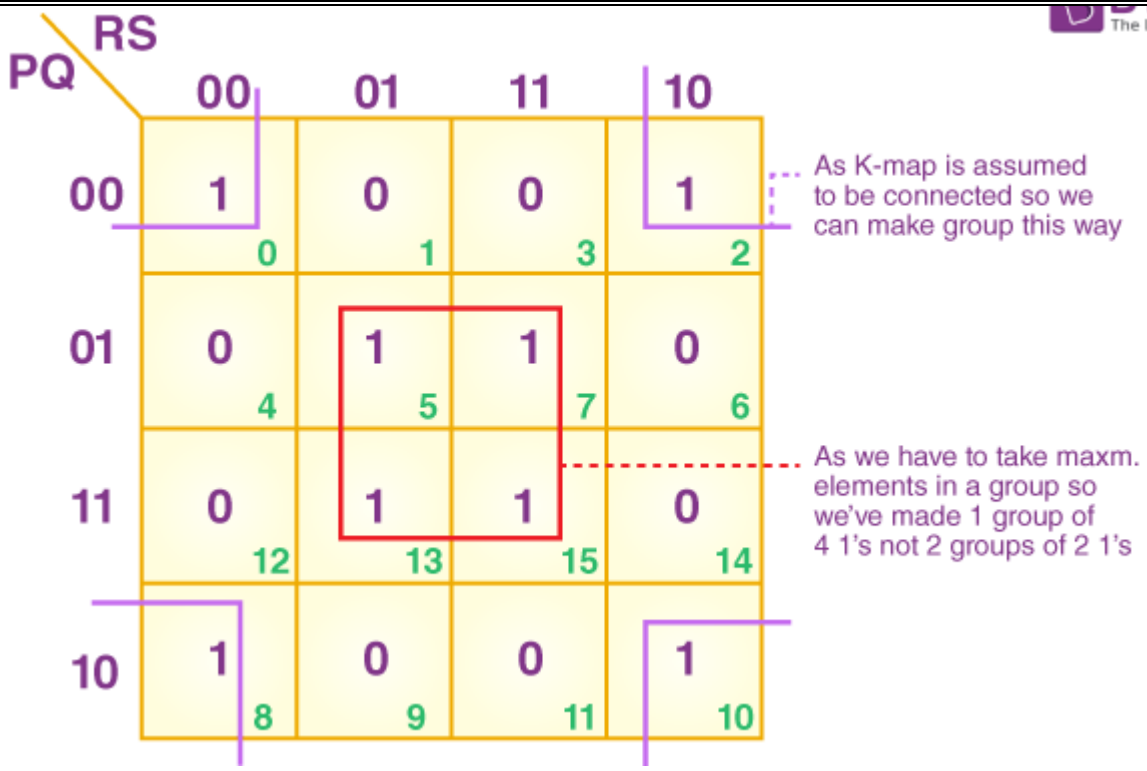
4 variables K-map:

General Syntax :



Example:

1) F (A, B, C, D) = ∑(0, 2, 5, 7, 8, 10, 13, 15)



From the red group, the product term would be —BD
From the lilac group, the product term would be —B'D'
If we sum these product terms, then we will get this final expression (BD + B'D')

Product of Sum Form(POS)

The POS stands for Product of Sum defined as the product of maxterms. As the name suggests the POS form represents the AND operation of the sum terms i.e., maxterms. The POS represents the maxterms in Boolean algebra. The POS is denoted by Π. As the POS deals with the maxterms it works on active low logic

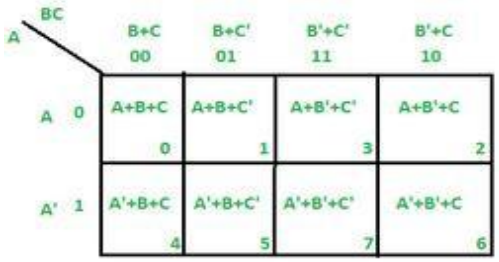
(A+B).(A+B'+C).(A+B)

Sum Sum Sum

Product Product

3-variables K map

General syntax:

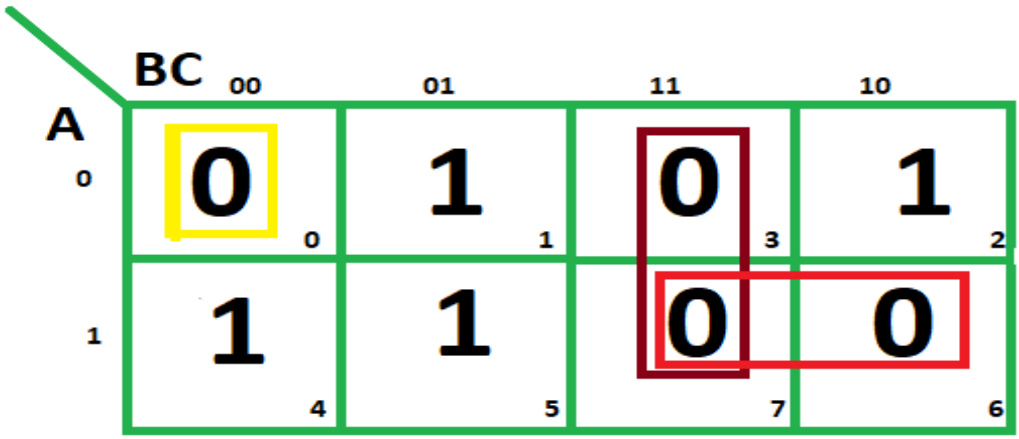


POS (MAXTERMS)

8 Blocks = 0
4 Blocks = 1 variable term
2 Blocks = 2 variable term
1 Block = 3 variable term

Example:

1) F(A,B,C)= π (0,3,6,7)



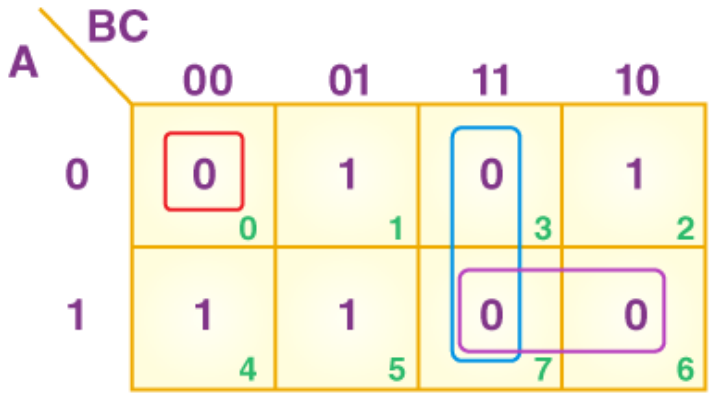
From **red** group we find terms: (A' + B') From **brown** group we find terms : (B'+C')

From **yellow** group we find terms : (A + B + C)

We will take product of these three terms :

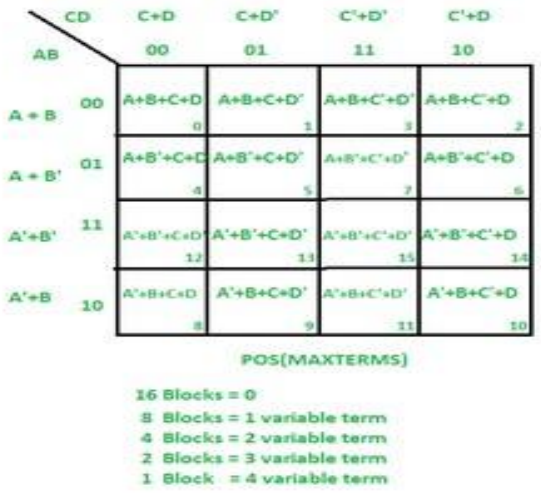
Final expression – (A' + B') (B' + C') (A + B + C)

2) F (P, Q, R) = π (0,3,6,7)

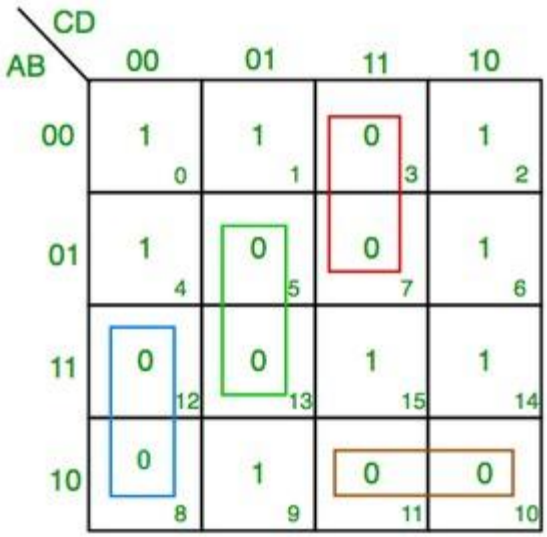


From the lilac group, the terms would be: P Q
If we take the complement of these two: P' Q'
And then sum up them:(P' + Q')
From the blue group, the terms would be: B R
When we take the complement of these terms : B' R' And
then sum them up: (B' + R')
From the red group, the terms would be : P' Q' R' If
we take the complement of the two terms: P Q R And
then sum them up: (P + Q + R)
If we take the product of these three terms, then we will get this
final expression –(P' + Q') (P' + R') (P + Q + R)

4-variables K map
General syntax:

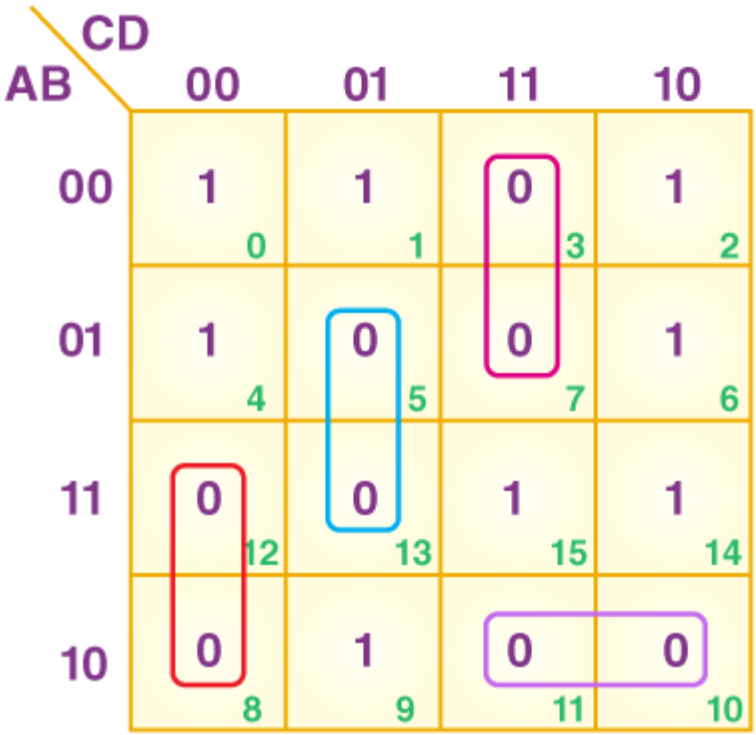


1) F(A,B,C,D)=∏(3,5,7,8,10,11,12,13)



From **green** group we find terms : (C+D'+B')
From **red** group we find terms : (C'+D'+A) From
blue group we find terms: (A'+C+D) From
brown group we find terms: (A'+B+C')
Finally we express these as product –**(C+D'+B').(C'+D'+A).(A'+C+D).(A'+B+C')**

2) F (P, Q, R, S) = π (3, 5, 7, 8, 10, 11, 12, 13)



From the blue group, the terms would be: R’ S Q
We take their complement and then sum them:
(R + S’+ Q’) From the purple group, the terms would be: R S P’
We take their complement and then sum them:
(R’ + S’+ P) S From the red group, the terms would be: P R’ S’
We take their complement and then sum them:
(P’ + R + S) From the lilac group, the terms would be: P Q’ R
We take their complement and then sum them: (P’ + Q + R’)
Finally –(R + S’+ Q’).(R’ + S’+A).(P’+ R + S).(P’+ Q + R’)

Difference Between SOP And POS

Characterization	**SOP**	**POS**
Definition	SOP is sum of the minterms.	POS is the product of maxterms.
Stands for	SOP stands for Sum of Product.	POS stands for Product of Sum.
Representation	It represents minterms.	It represents maxterms.
Logic	It works on active high logic.	It works on active low logic.
Denotation	It is denoted by Σ.	It is denoted by Π.
Output	The output of SOP is 1.	The output of POS is 0.